Universidad Nacional Autónoma de México
Escuela Nacional de Estudios Superiores, Unidad Juriquilla

Computadoras y Programación

Tarea 7 / Ejercicio 7

Subdiagramas y diagramas estructurados

Alumno: Daniel Rodríguez Correa

Fecha: 29 de octubre de 2026

Ejercicio 1. Paquetería: intentos de entrega

Herramienta: diagrama de flujo

Análisis

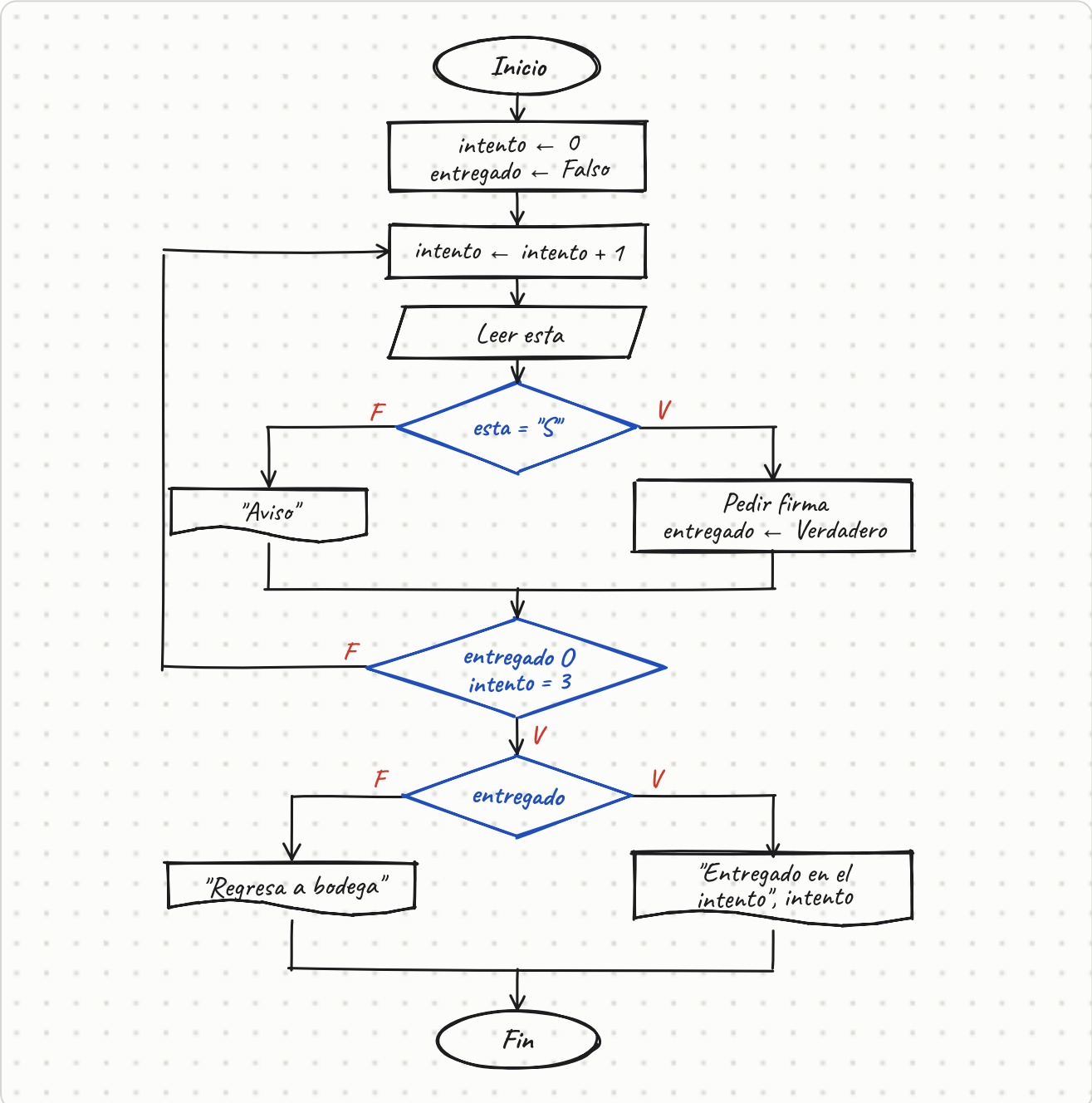
Entradas: en cada intento, si el cliente está o no (S/N).

Proceso: uso un ciclo Repetir. En cada vuelta sumo un intento y pregunto si el cliente está. Si está, se pide su firma y el paquete queda entregado; si no, se deja un aviso. El ciclo se detiene cuando ya se entregó o cuando ya van 3 intentos. Al salir reviso cuál de las dos cosas pasó.

Salidas: "Aviso" en cada intento fallido y al final "Entregado en el intento n" o "Regresa a bodega".

¿Por qué diagrama de flujo? El diagrama es para capacitar a un repartidor nuevo. Con las flechas se ve el orden de los pasos y el regreso al siguiente intento, y no hace falta saber programar para seguirlo.

Diagrama de flujo



Programa en PSeInt

```
Proceso Paqueteria
    Definir intento Como Entero;
    Definir esta Como Caracter;
    Definir entregado Como Logico;
    intento <- 0;
    entregado <- Falso;
    Repetir
        intento <- intento + 1;
        Escribir "Intento ", intento, ". ¿Está el cliente? (S/N):";
        Leer esta;
        Si esta = "S" Entonces
            Escribir "Firma recibida";
            entregado <- Verdadero;
        SiNo
            Escribir "Aviso";
        FinSi
    Hasta Que entregado O intento = 3
    Si entregado Entonces
        Escribir "Entregado en el intento ", intento;
    SiNo
        Escribir "Regresa a bodega";
    FinSi
FinProceso
```

Prueba de escritorio

¿Está el cliente?	Salida
N, S	Aviso, Entregado en el intento 2
S	Entregado en el intento 1
N, N, N	Aviso (3 veces), Regresa a bodega

Ejercicio 2. Videojuego: adivina el número

Herramienta: diagrama de flujo

Análisis

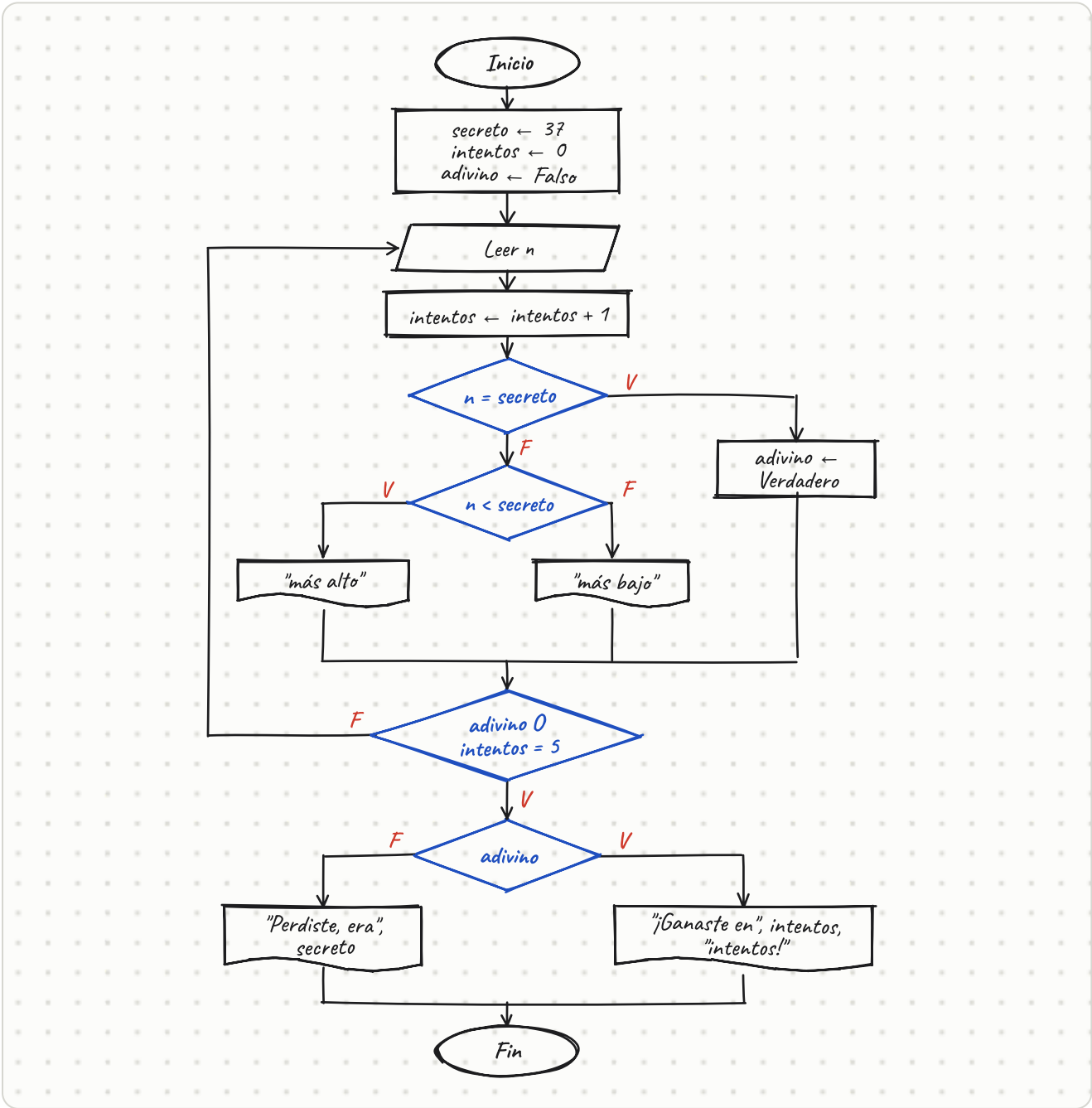
Entradas: el número que escribe el jugador en cada intento (el secreto es 37).

Proceso: con un Repetir leo el número y cuento el intento. Lo comparo con el secreto: si es igual, adivinó; si es menor, el juego dice "más alto"; si es mayor, "más bajo". Termina cuando adivina o cuando llega a 5 intentos.

Salidas: la pista de cada intento y al final "¡Ganaste en n intentos!" o "Perdiste, era 37".

¿Por qué diagrama de flujo? El juego es una serie de turnos que se repite. En el diagrama de flujo se ven bien el regreso al siguiente intento, los tres caminos de la comparación y las dos formas de terminar.

Diagrama de flujo



Programa en PSeInt

```
Proceso AdivinaNumero
    Definir secreto, n, intentos Como Entero;
    Definir adivino Como Logico;
    secreto <- 37;
    intentos <- 0;
    adivino <- Falso;
    Repetir
        Escribir "Intento ", intentos + 1, " de 5. Tu número:";
        Leer n;
        intentos <- intentos + 1;
        Si n = secreto Entonces
            adivino <- Verdadero;
        SiNo
            Si n < secreto Entonces
                Escribir "más alto";
            SiNo
                Escribir "más bajo";
            FinSi
        FinSi
    Hasta Que adivino O intentos = 5
    Si adivino Entonces
        Escribir "¡Ganaste en ", intentos, " intentos!";
    SiNo
        Escribir "Perdiste, era ", secreto;
    FinSi
FinProceso
```

Prueba de escritorio

Intentos	Salida
50, 25, 37	más bajo, más alto, ¡Ganaste en 3 intentos!
10, 20, 30, 40, 45	más alto (3 veces), más bajo (2 veces), Perdiste, era 37

Ejercicio 3. Mundial: goles de un grupo

Herramienta: diagrama Nassi-Shneiderman

Análisis

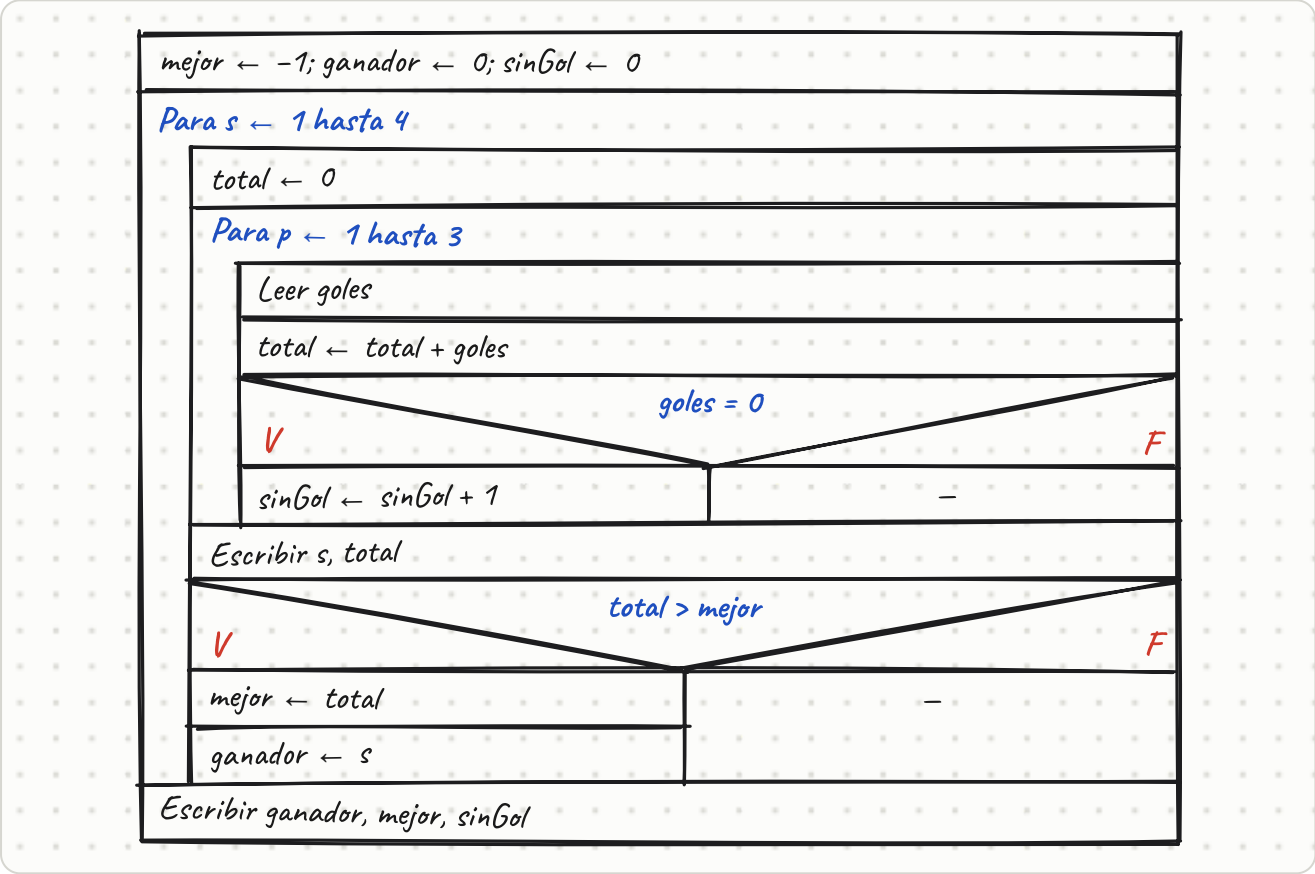
Entradas: los goles de cada selección en cada partido (4 selecciones por 3 partidos, 12 datos).

Proceso: un Para recorre las selecciones y otro Para, adentro, recorre sus 3 partidos. El total de goles se pone en 0 al empezar cada selección; el contador de partidos sin gol no se reinicia porque es de todo el grupo. Al terminar cada selección comparo su total con el mejor que llevo (mejor empieza en −1 para que cualquier total cuente, incluso 0).

Salidas: el total de cada selección, la que metió más goles y cuántos partidos terminaron sin gol.

¿Por qué Nassi-Shneiderman? Hay dos ciclos, uno dentro del otro. Con las cajas se ve en qué nivel va cada contador y cuál se reinicia, que es justo lo que hay que cuidar al programarlo.

Diagrama Nassi-Shneiderman



Programa en PSeInt

```
Proceso GolesGrupo
    Definir s, p, goles, total, mejor, ganador, sinGol Como Entero;
    mejor <- -1;
    ganador <- 0;
    sinGol <- 0;
    Para s <- 1 Hasta 4 Con Paso 1 Hacer
        total <- 0;
        Para p <- 1 Hasta 3 Con Paso 1 Hacer
            Escribir "Selección ", s, ", partido ", p, ". Goles:";
            Leer goles;
            total <- total + goles;
            Si goles = 0 Entonces
                sinGol <- sinGol + 1;
            FinSi
        FinPara
        Escribir "Selección ", s, ": ", total, " goles";
        Si total > mejor Entonces
            mejor <- total;
            ganador <- s;
        FinSi
    FinPara
    Escribir "Más goles: selección ", ganador, " (", mejor, ")";
    Escribir "Partidos sin gol: ", sinGol;
FinProceso
```

Prueba de escritorio

Selección: goles	Total
1: 2, 1, 0	3
2: 0, 0, 1	1
3: 3, 2, 2	7
4: 1, 0, 0	1
Al final	Más goles: selección 3 (7). Partidos sin gol: 5

Ejercicio 4. Supermercado: menú de la caja

Herramienta: diagrama Nassi-Shneiderman (Repetir con Segun)

Análisis

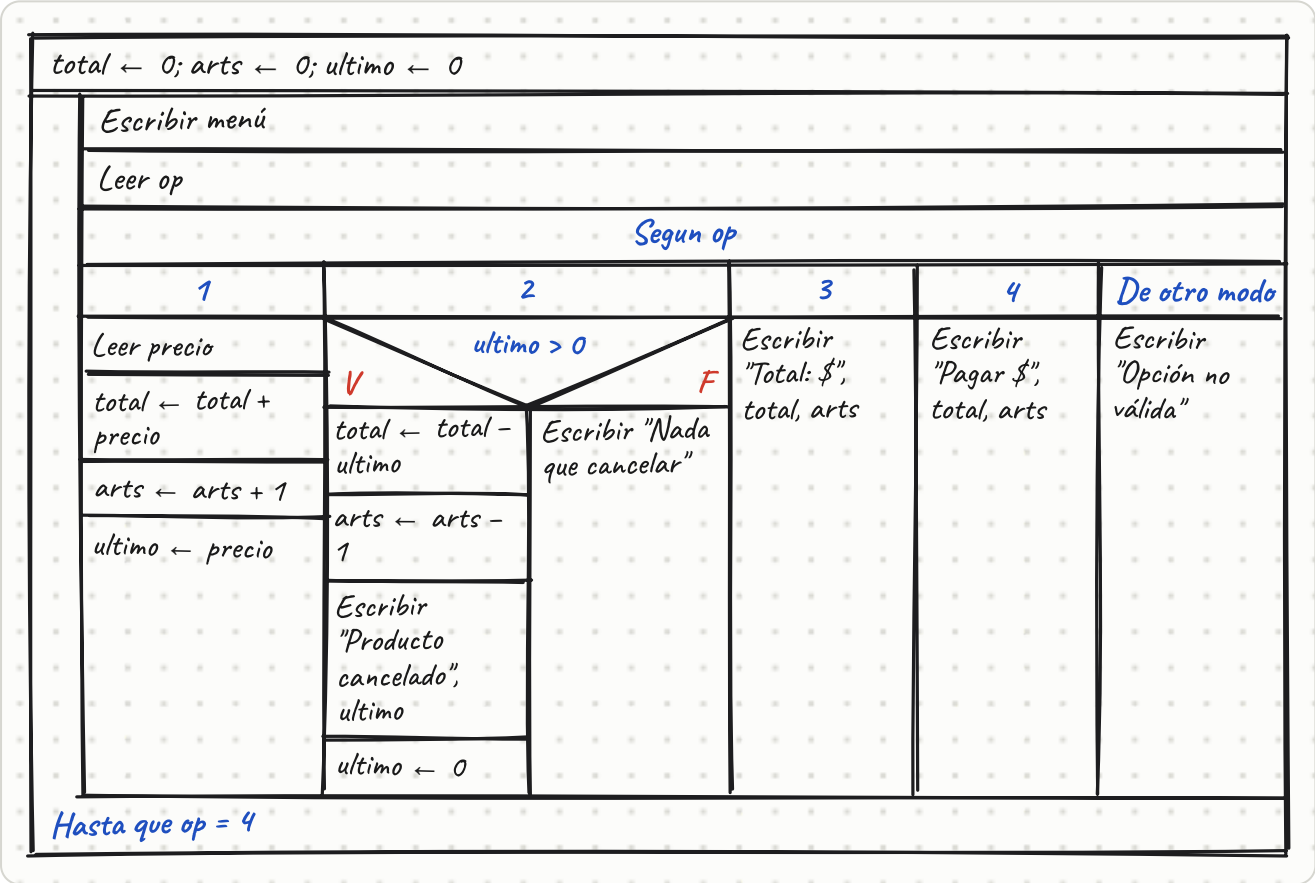
Entradas: la opción del menú y, si es la 1, el precio del producto.

Proceso: un Repetir con un Segun adentro. Guardo el precio del último producto en la variable ultimo. Al cancelar resto ese precio y pongo ultimo en 0, así ya no se puede cancelar otra vez. Si ultimo vale 0 se muestra "Nada que cancelar".

Salidas: el mensaje de cada opción, el total con el número de artículos y lo que hay que pagar.

¿Por qué Nassi-Shneiderman? Tengo un Segun dentro de un Repetir y además un Si dentro de uno de los casos. En N-S cada estructura es una caja dentro de otra y se pasa casi directo al código.

Diagrama Nassi-Shneiderman



Programa en PSeInt

```
Proceso MenuCaja
    Definir op, arts Como Entero;
    Definir precio, total, ultimo Como Real;
    total <- 0;
    arts <- 0;
    ultimo <- 0;
    Repetir
        Escribir "1 Agregar  2 Cancelar último  3 Ver total  4 Pagar y salir";
        Leer op;
        Segun op Hacer
            1:
                Escribir "Precio:";
                Leer precio;
                total <- total + precio;
                arts <- arts + 1;
                ultimo <- precio;
            2:
                Si ultimo > 0 Entonces
                    total <- total - ultimo;
                    arts <- arts - 1;
                    Escribir "Producto cancelado (el de $", ultimo, ")";
                    ultimo <- 0;
                SiNo
                    Escribir "Nada que cancelar";
                FinSi
            3:
                Escribir "Total: $", total, ", artículos: ", arts;
            4:
                Escribir "Pagar $", total, ", artículos: ", arts;
            De Otro Modo:
                Escribir "Opción no válida";
        FinSegun
    Hasta Que op = 4
FinProceso
```

Prueba de escritorio

Opciones (en orden)	Salida
1 (50), 1 (30), 2	Producto cancelado (el de $30)
2	Nada que cancelar
3	Total: $50, artículos: 1
4	Pagar $50, artículos: 1

Ejercicio 5. Laboratorio: la computadora no enciende

Herramienta: árbol de decisión

Análisis

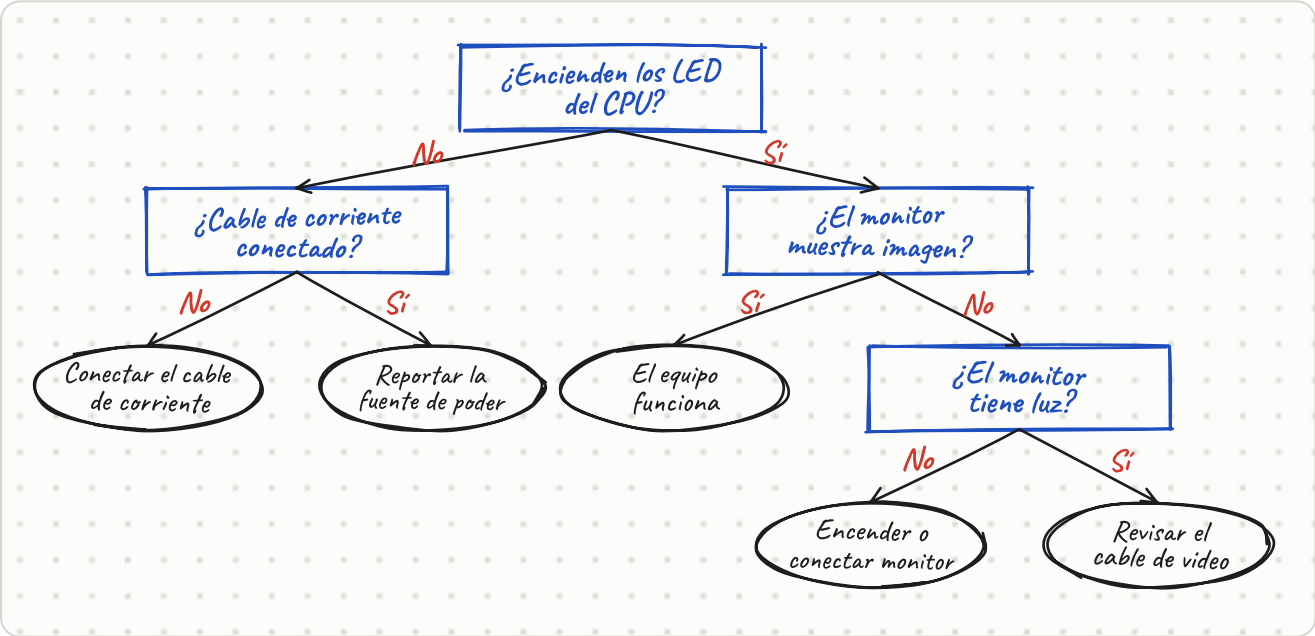
Entradas: respuestas Sí/No: si encienden los LED del CPU, si el cable de corriente está conectado, si el monitor muestra imagen y si tiene luz.

Proceso: cada revisión depende de la anterior. La raíz es lo primero que se revisa en la realidad, los LED. Antes de dibujar conté las acciones posibles: son 5, así que el árbol tiene **5 hojas** (y 4 preguntas).

Salidas: una sola indicación para el personal.

¿Por qué árbol de decisión? Es un diagnóstico: importa qué se revisa primero y qué después. Además se evitan preguntas que no sirven; por ejemplo, si los LED no encienden ya no se revisa el monitor.

Árbol de decisión (5 hojas)



Programa en PSeInt (solo hace las preguntas necesarias)

```
Proceso DiagnosticoPC
    Definir led, cable, imagen, luz Como Caracter;
    Escribir "¿Encienden los LED del CPU? (S/N):";
    Leer led;
    Si led = "N" Entonces
        Escribir "¿El cable de corriente está conectado? (S/N):";
        Leer cable;
        Si cable = "N" Entonces
            Escribir "Conectar el cable de corriente";
        SiNo
            Escribir "Reportar la fuente de poder";
        FinSi
    SiNo
        Escribir "¿El monitor muestra imagen? (S/N):";
        Leer imagen;
        Si imagen = "S" Entonces
            Escribir "El equipo funciona";
        SiNo
            Escribir "¿El monitor tiene luz? (S/N):";
            Leer luz;
            Si luz = "N" Entonces
                Escribir "Encender o conectar el monitor";
            SiNo
                Escribir "Revisar el cable de video";
            FinSi
        FinSi
    FinSi
FinProceso
```

Prueba de escritorio

Respuestas	Indicación
LED no, cable no	Conectar el cable de corriente
LED no, cable sí	Reportar la fuente de poder
LED sí, imagen sí	El equipo funciona
LED sí, imagen no, luz no	Encender o conectar el monitor
LED sí, imagen no, luz sí	Revisar el cable de video

Ejercicio 6. Rutas: ¿cómo llego?

Herramienta: árbol de decisión

Análisis

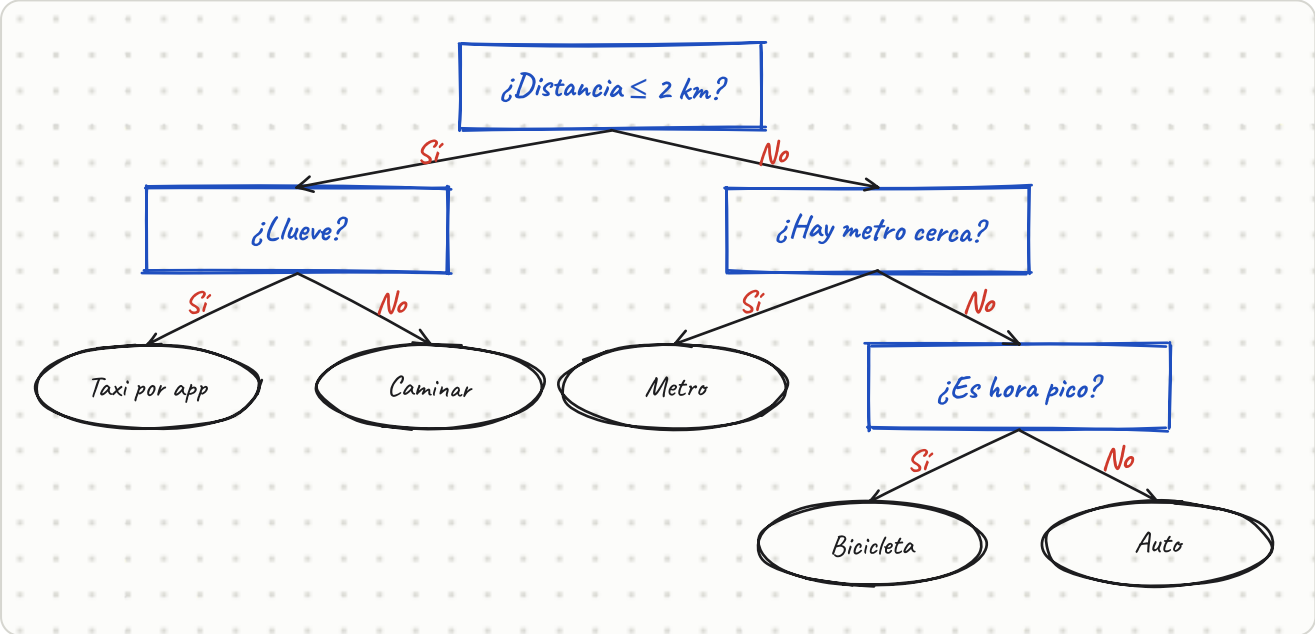
Entradas: la distancia en km, si llueve, si hay metro cerca y si es hora pico.

Proceso: la distancia va en la raíz porque de ella depende lo que se pregunta después. La lluvia solo importa cuando son 2 km o menos; el metro y la hora pico, solo cuando son más de 2 km.

Salidas: una recomendación: taxi por app, caminar, metro, bicicleta o auto.

¿Por qué árbol de decisión? Las preguntas dependen de la respuesta anterior. En el árbol se ve que nunca se pregunta por la lluvia en un viaje largo ni por el metro en uno corto.

Árbol de decisión (5 hojas)



Programa en PSeInt

```
Proceso RecomendarRuta
    Definir dist Como Real;
    Definir llueve, metro, pico Como Caracter;
    Escribir "Distancia al destino (km):";
    Leer dist;
    Si dist <= 2 Entonces
        Escribir "¿Llueve? (S/N):";
        Leer llueve;
        Si llueve = "S" Entonces
            Escribir "Taxi por app";
        SiNo
            Escribir "Caminar";
        FinSi
    SiNo
        Escribir "¿Hay metro cerca? (S/N):";
        Leer metro;
        Si metro = "S" Entonces
            Escribir "Metro";
        SiNo
            Escribir "¿Es hora pico? (S/N):";
            Leer pico;
            Si pico = "S" Entonces
                Escribir "Bicicleta";
            SiNo
                Escribir "Auto";
            FinSi
        FinSi
    FinSi
FinProceso
```

Prueba de escritorio

Entrada	Recomendación
1.5 km, llueve	Taxi por app
1.5 km, no llueve	Caminar
8 km, metro sí	Metro
8 km, metro no, hora pico	Bicicleta
8 km, metro no, no es hora pico	Auto

Ejercicio 7. Rutas: tarifa de una app de transporte

Herramienta: tabla de decisión

Análisis

Entradas: la tarifa base, los kilómetros del viaje, si es hora pico y si llueve.

Proceso: con 3 condiciones salen $2^3 = 8$ reglas. Si hay hora pico y lluvia, la tarifa se multiplica por 1.5; si solo hay una de las dos, por 1.2; si no hay ninguna, se cobra normal. Aparte, si el viaje es de más de 10 km se suman $30.

Salidas: la tarifa final.

¿Por qué tabla de decisión? Es una regla de cobro y no puede quedar ningún caso sin tarifa. En la tabla están las 8 combinaciones y se ve que ninguna aplica dos multiplicadores al mismo tiempo.

Tabla de decisión completa (8 reglas)

Condiciones	R1	R2	R3	R4	R5	R6	R7	R8
C1 ¿Hora pico?	S	S	S	S	N	N	N	N
C2 ¿Llueve?	S	S	N	N	S	S	N	N
C3 ¿Viaje > 10 km?	S	N	S	N	S	N	S	N
Acciones								
A1 Tarifa × 1.5	X	X						
A2 Tarifa × 1.2			X	X	X	X		
A3 Tarifa normal							X	X
A4 Cargo de $30	X		X		X		X	

¿Se puede simplificar?

No. Para unir dos reglas tienen que tener las mismas acciones y ser diferentes en una sola condición. Las parejas que solo cambian en C3 (R1 y R2, R3 y R4, R5 y R6, R7 y R8) no se pueden unir porque cambia el cargo de $30. Las que solo cambian en C1 o en C2 tienen distinto multiplicador. Las únicas reglas con las mismas acciones son R3 (S N S) y R5 (N S S), pero son diferentes en dos condiciones, así que tampoco se unen. La tabla se queda con sus 8 reglas.

Programa en PSeInt

```
Proceso TarifaTransporte
    Definir tarifa, km Como Real;
    Definir pico, lluvia Como Caracter;
    Escribir "Tarifa base, km, ¿hora pico? (S/N), ¿llueve? (S/N):";
    Leer tarifa, km, pico, lluvia;
    Si pico = "S" Y lluvia = "S" Entonces
        tarifa <- tarifa * 1.5;
    SiNo
        Si pico = "S" O lluvia = "S" Entonces
            tarifa <- tarifa * 1.2;
        FinSi
    FinSi
    // el cargo va aparte porque se suma en cualquier caso
    Si km > 10 Entonces
        tarifa <- tarifa + 30;
    FinSi
    Escribir "Tarifa: $", tarifa;
FinProceso
```

El multiplicador lo hice con Si anidados porque solo se puede aplicar uno (A1, A2 o A3). El cargo de $30 es un Si separado porque se puede sumar a cualquiera.

Prueba de escritorio (tarifa base de $100)

C1 C2 C3	Acciones	Tarifa
S S S	×1.5 y cargo	$180
S N N	×1.2	$120
N S S	×1.2 y cargo	$150
N N N	tarifa normal	$100

Ejercicio 8. Videojuego: ¿qué hace el héroe?

Herramienta: tabla de decisión

Análisis

Entradas: la vida del héroe, si tiene poción y si el enemigo es jefe.

Proceso: 3 condiciones, 8 reglas. Con vida menor que 30: si tiene poción la usa; si no tiene, huye del jefe y a los demás los ataca. Con vida de 30 o más siempre ataca y, si es jefe, también activa el escudo.

Salidas: lo que hace el héroe en ese turno.

¿Por qué tabla de decisión? Son las reglas de comportamiento del personaje. Con la tabla me aseguro de que ningún turno se quede sin acción y después la puedo reducir.

Tabla de decisión completa (8 reglas)

Condiciones	R1	R2	R3	R4	R5	R6	R7	R8
C1 ¿Vida < 30?	S	S	S	S	N	N	N	N
C2 ¿Tiene poción?	S	S	N	N	S	S	N	N
C3 ¿Enemigo jefe?	S	N	S	N	S	N	S	N
Acciones								
A1 Usar poción	X	X						
A2 Huir			X					
A3 Activar escudo					X		X	
A4 Atacar				X	X	X	X	X

Tabla simplificada (5 reglas)

Condiciones	R1-2	R3	R4	R5-7	R6-8
C1 ¿Vida < 30?	S	S	S	N	N
C2 ¿Tiene poción?	S	N	N	–	–
C3 ¿Enemigo jefe?	–	S	N	S	N
Acciones					
A1 Usar poción	X				
A2 Huir		X			
A3 Activar escudo				X	
A4 Atacar			X	X	X

R1 y R2 tienen la misma acción (usar poción) y solo cambian en C3, así que C3 queda con "—": si tiene poción no importa si es jefe. R5 y R7 hacen lo mismo (escudo y atacar) y solo cambian en C2. R6 y R8 solo atacan y también cambian solo en C2. De 8 reglas quedan 5 y el héroe se comporta igual.

Programa en PSeInt

```
Proceso TurnoHeroe
    Definir vida Como Entero;
    Definir pocion, jefe Como Caracter;
    Escribir "Vida, ¿tiene poción? (S/N), ¿enemigo jefe? (S/N):";
    Leer vida, pocion, jefe;
    Si vida < 30 Entonces
        Si pocion = "S" Entonces
            Escribir "Usar poción";
        SiNo
            Si jefe = "S" Entonces
                Escribir "Huir";
            SiNo
                Escribir "Atacar";
            FinSi
        FinSi
    SiNo
        Si jefe = "S" Entonces
            Escribir "Activar escudo";
        FinSi
        Escribir "Atacar";
    FinSi
FinProceso
```

Prueba de escritorio

C1 C2 C3	Acciones
S S N	usar poción
S N S	huir
S N N	atacar
N S S	activar escudo y atacar

Ejercicios 9 y 10. ¿Qué herramienta usarías?

Para elegir me fijé en qué pregunta tiene que responder cada caso: el diagrama de flujo responde en qué orden pasan las cosas, el N-S cómo se anidan las estructuras, el árbol qué se pregunta primero y la tabla si están cubiertos todos los casos.

Ejercicio 9

Situación	Herramienta	Por qué	Por qué no otra
A. Supermercado: explicar a cajeros nuevos la devolución, paso por paso.	**Diagrama de flujo**	Es un proceso en orden y lo van a leer personas que no programan. Con las flechas se sigue paso por paso.	El N-S es más para programar. El árbol y la tabla solo muestran decisiones, no toda la secuencia.
B. Paquetería: revisar que las comisiones (4 condiciones Sí/No) no dejen casos sin pago.	**Tabla de decisión**	Obliga a escribir las $2^4 = 16$ combinaciones, así se ve si falta un caso o si dos reglas se contradicen.	Con un árbol es fácil saltarse combinaciones. Flujo y N-S muestran el proceso, pero no comprueban que estén todos los casos.
C. Laboratorio: diagnosticar por qué no imprime; cada revisión depende de la anterior.	**Árbol de decisión**	Ordena las revisiones y en cada camino solo se pregunta lo necesario.	La tabla pierde el orden y obliga a poner combinaciones que no aplican.
D. Videojuego: diseñar antes de programar tres ciclos anidados (filas, columnas, capas).	**Nassi-Shneiderman**	Son tres cajas Para, una dentro de otra, sin flechas, y se pasan directo al código.	En diagrama de flujo, tres ciclos anidados llenan el dibujo de flechas de regreso. Árbol y tabla no tienen ciclos.

Ejercicio 10

Situación	Herramienta	Por qué	Por qué no otra
E. Mundial: clasificar por lugar y, solo si hace falta, por diferencia de goles.	**Árbol de decisión**	El lugar va en la raíz y la diferencia de goles solo aparece en las ramas donde se necesita.	La tabla no muestra el orden de las preguntas ni el "solo si hace falta".
F. Casino: documentar el menú del simulador (Repetir, Segun y Si) para pasarlo a C.	**Nassi-Shneiderman**	El Repetir, el Segun y el Si quedan como cajas anidadas que se pasan casi línea por línea a C.	Un diagrama de flujo con un Segun de varios casos dentro de un ciclo se llena de flechas. Árbol y tabla no tienen ciclos.
G. Tienda: comprobar que las promociones del Buen Fin no se contradicen.	**Tabla de decisión**	En cada columna están todas las promociones de una combinación, así que dos descuentos incompatibles en la misma regla se notan luego luego.	En un árbol los casos quedan repartidos en ramas y no se ven dos acciones juntas.
H. Rutas: presentar a los usuarios cómo se calcula una ruta (pedir origen, buscar caminos, elegir el más corto, mostrarlo).	**Diagrama de flujo**	Es una secuencia de pasos para gente que no programa.	El N-S es muy técnico para usuarios. Árbol y tabla muestran decisiones, no un proceso.